Classical Circuit Calculator

Iteration 02: System Design and Implementation Start
Eric Sebastian and Joshua Marquez
October 30, 2026

1. Scope and leadership

Our project is a simple C++ calculator for basic DC resistive circuits. It calculates resistance, current, power, and equivalent resistance for series and parallel resistors. We use a text menu and show units with each result. AC circuits and complicated networks are outside our scope.

Joshua Marquez is the leader for this iteration because Eric led Iteration 1. Joshua will coordinate progress, keep the task table current, check that the work is shared fairly, and present the check-in status.

2. Starting point and current progress

The team had not written code before this step. An assisted starter implementation is now prepared with all five calculations, a repeating switch menu, input checks, and 16 automated test cases. The starter compiled and all 16 cases passed in the preparation environment. Each member still needs to review their assigned code and record their actual contribution. GitHub commits, peer reviews, and submission have not been verified.

3. Menu and input rules

Option	Task	Rule
1	Resistance, R = V / I	Voltage >= 0; current > 0
2	Current, I = V / R	Voltage >= 0; resistance > 0
3	Power, P = V x I	Voltage and current >= 0
4	Series, Req = sum of R	1-100 positive resistors
5	Parallel, Req = 1 / sum(1/R)	1-100 positive resistors
0	Exit	Ends program

Inputs use volts, amperes, and ohms. Each prompt accepts one finite number. Invalid entries return to the menu. Outputs use four decimal places. This version uses nonnegative magnitudes and does not model ideal zero-ohm resistors.

4. Task table

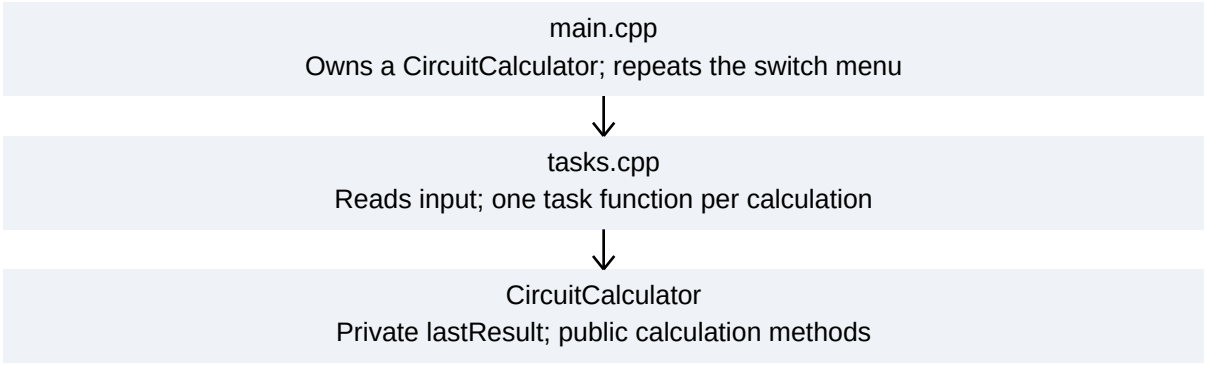
The names below identify assigned responsibility. In progress means starter code exists and student review is pending. Non-menu support work is listed separately from calculation features.

ID / menu	Task / owner	Function and deliverable	Status
T1 / 1	Resistance Eric	resistanceTask / resistance Working R=V/I with ohms	in progress
T2 / 2	Current Eric	currentTask / current Working I=V/R with A	in progress
T3 / 3	Power Joshua	powerTask / power Working P=VI with W	in progress
T4 / 4	Series resistance Eric	seriesTask / series Sum 1-100 resistors	in progress
T5 / 5	Parallel resistance Joshua	parallelTask / parallel Reciprocal resistor sum	in progress
T6 / 0-5	Menu and exit Joshua	main.cpp / readMenuChoice Repeating switch menu	in progress
T7 / N/A	Testing Both	test.sh / tests/ 16 starter cases; team review pending	in progress
T8 / N/A	Documentation Both	README.md / docs/ PDF and Excel starter; team review pending	in progress
T9 / N/A	Review and submit Both	GitHub / Canvas One review each, commits, individual links	not started

T1-T5 use src/tasks.cpp for prompts and src/calculator.cpp for formulas, with declarations in tasks.h and calculator.h. Eric owns T1, T2, and T4. Joshua owns T3, T5, and the menu. Both share testing, documentation, and review.

5. Class design and file organization

CircuitCalculator stores one private data member, **double lastResult**, initialized to zero. The private **save(double)** method checks and stores a finite answer. Public methods are resistance(voltage, current), current(voltage, resistance), power(voltage, current), series(resistors), parallel(resistors), and getLastResult() const. Resistor lists use vector<double>. Calculation methods return double and report invalid values with an exception.



File	Purpose
src/main.cpp	Main loop, switch menu, and exception handling
src/tasks.h / tasks.cpp	Task declarations, prompts, and input validation
src/calculator.h / calculator.cpp	Class declaration, private state, and formulas
tests/ and test.sh	Saved inputs, expected output lines, automated runner
README.md	Build, usage, scope, and team workflow
docs/	This report and the Excel project plan

6. Implementation and testing

Each switch case calls one task function. The task reads the values, calls the calculator method, and prints the result with units. The menu repeats until option 0 or end of input. All calculation features have working starter implementations, so no unfinished calculation stubs remain.

The 16 tests cover all five formulas, decimal rounding, zero power, zero denominators, negative resistance, text, invalid menu choices, fractional resistor counts, extra text, overflow, and repeated calculations. The Bash script builds with C++17 and warning flags, checks expected output text, and exits nonzero on failure. Team members should run the tests on their own computers and add cases during review.

7. Revised timeline and team contributions

Order	Milestone	Owner
1	Review the design and starter code	Both
2	Review assigned calculation functions	Eric: R, I, series; Joshua: P, parallel, menu
3	Run tests and improve cases	Both; test each other's features
4	Complete one code review per person	Both
5	Update actual contributions and documentation	Both
6	Commit source, README, and docs to GitHub	Both
7	Submit repository link separately on Canvas	Each student

The assignment screenshot shows availability ending October 30 at 11:59 PM. Earlier milestone dates were not provided, so the timeline records the remaining order of work instead of inventing past completion dates. Future iteration dates remain TBD.

Member	Actual contribution record	Next responsibility
Eric Sebastian	Pending Eric's entry in the Excel plan	Review resistance, current, and series; tests and examples; review Joshua's code
Joshua Marquez	Pending Joshua's entry in the Excel plan	Lead this iteration; review menu, power, and parallel; setup and usage; review Eric's code

Assigned work is not a claim that a student already wrote or reviewed the assisted starter. Each person should replace the pending entry with specific work they actually complete.

8. Build and submission checklist

Build: g++ -std=c++17 -Wall -Wextra -pedantic src/*.cpp -o build/calculator
Run: ./build/calculator
Tests: bash test.sh
Create the build folder first. README.md also includes Windows PowerShell commands.

Before submission: review the code together, record actual contributions, complete peer reviews, and commit regularly with clear messages. Verify that GitHub contains src/, tests/, test.sh, README.md, and docs/ with this PDF and the Excel plan. Each student then submits the repository link on Canvas. These external actions remain pending.

Sources: Classical Circuit Calculator Iteration 1 project plan; Iteration 2 assignment screenshot; team details supplied in this conversation.